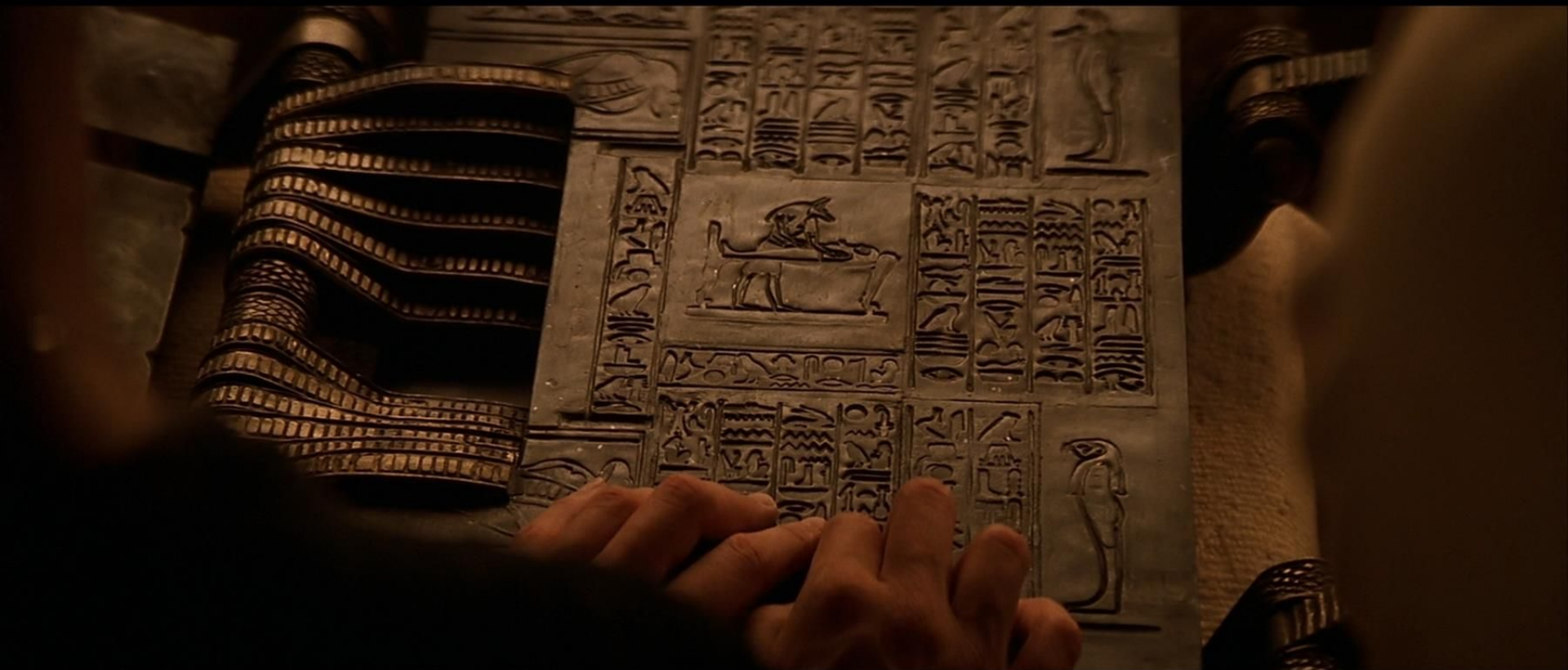


Технологии Программирования



Законы Древних

Время пришло!

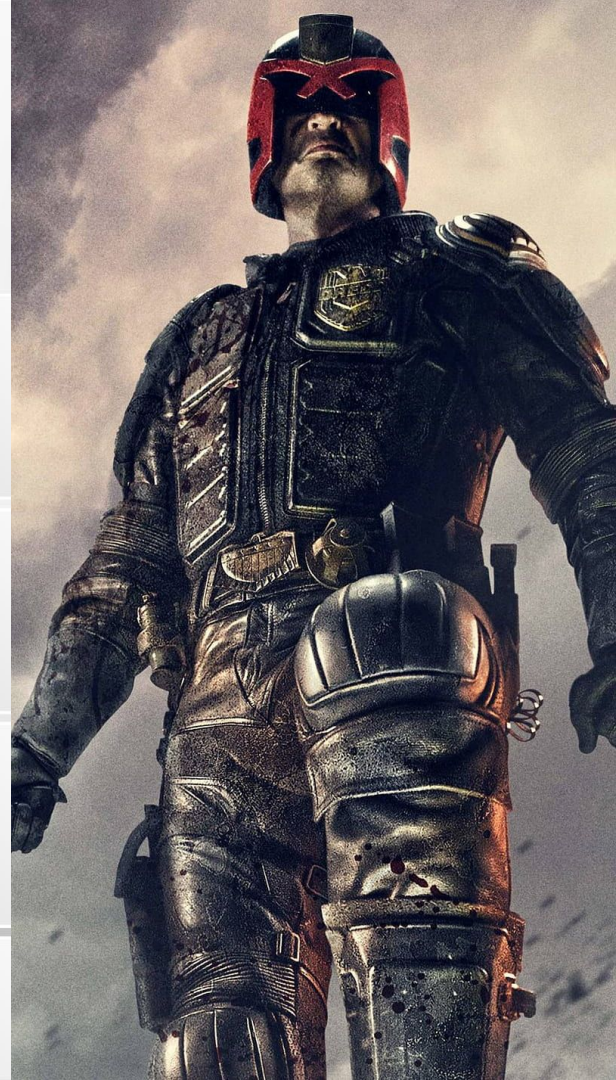
Задание.

- Отчет по проделанной работе в формате 1x1x1.
- Совет: не надо тратить больше 20 минут на отчет.

Изменение условий всех задач:

Треугольники -> Выпуклые многоугольники

1. **Ввод мышкой:** конец многоугольника - попадание мышкой в 1ю точку.
2. **Рандом:** количество вершин - произвольное (не больше 10).
3. **Дешифратор:** в файле возможны кроме "03" ключевые числа могут быть любыми от 03 до 09.
4. **Парсер:** функция triangle заменяется на polygon(N, x0, y0 ...) где N - количество вершин.

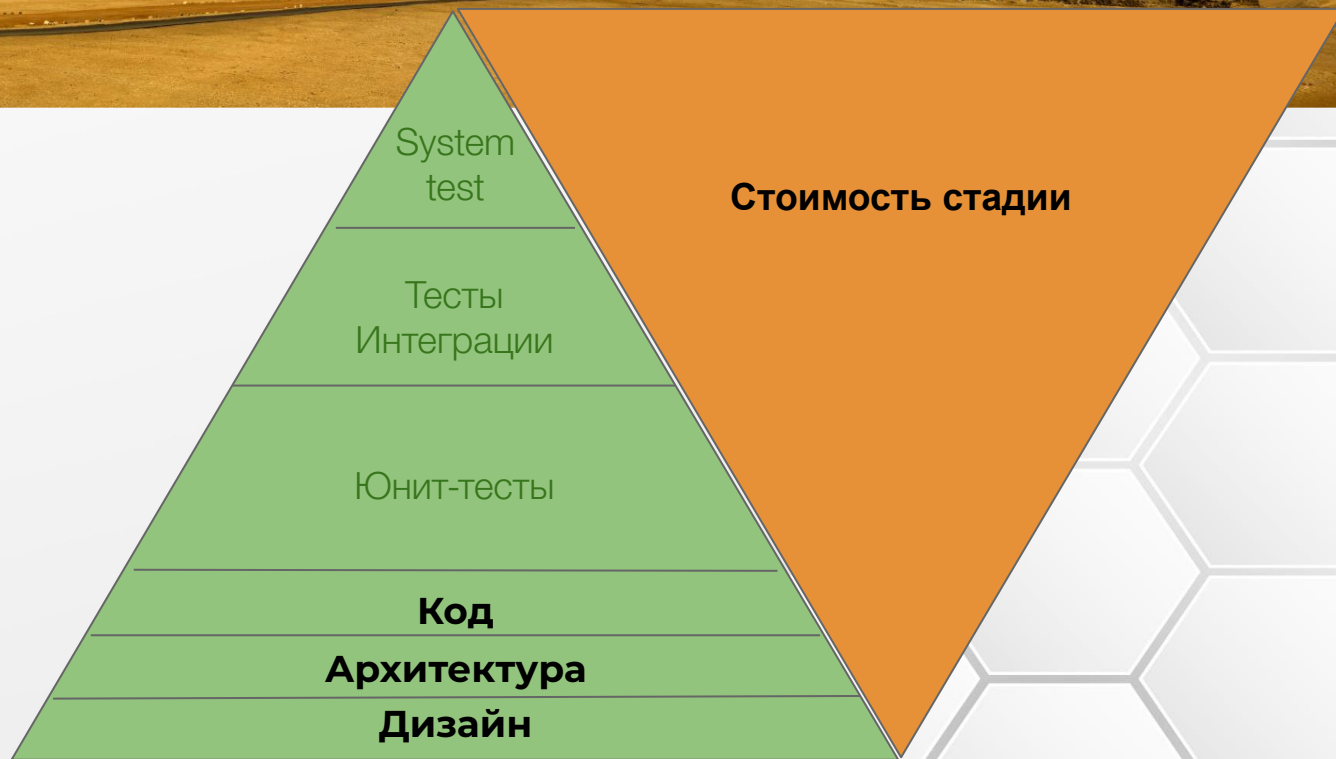


Вопрос!

Что делает следующая функция:

```
void getCoordinates(  
    std::vector<std::vector<double>> coordinates1,  
    std::vector<std::vector<double>> coordinates2,  
    std::vector<std::vector<double>>& output)  
{  
    std::vector<double> vertex1;  
    std::vector<double> vertex2;  
    std::vector<double> vertex3;  
    std::vector<double> vertex4;  
    std::vector<std::vector<double>> next;  
    output = coordinates1;  
    std::vector<std::vector<double>> new_coordinates;
```

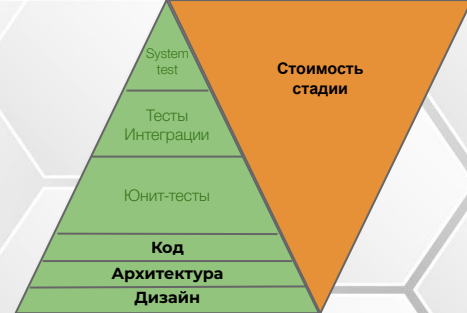
Пирамида



BDUF

Big Design Up Front

- Прежде чем писать - подумай.
- Перед началом разработки необходимо составить план, определить общую архитектуру продукта и взаимодействия между его элементами.
- Стоимость ошибок найденных на этапе планирования и архитектуры самая низкая, так как не требует исправления непосредственно кода.
- Важно соблюдать баланс между детальностью проработки и сложностью задачи, не надо продумывать все до деталей реализации и строчек кода.



DRY

Do not repeat yourself.

- Принцип: «Не повторяйся».
- Декларирует необходимость ухода от дублирующего кода (codedups)
 - Code duplicates - источник сложных ошибок, когда ошибка присутствует во всех “инстансах кода”, то количество исправлений должно быть равно количеству вхождений данного кода,
 - Если хоть в одном code duplicate остается баг, автор подсознательно будет искать ошибку в другом месте, эффект déjà vu: “этот код я же смотрел”,
 - Если в коде есть два повторяющихся участка, их необходимо перенести в метод, набор методов или лямбду,
 - Код необходимый в разных модулях программы обычно группируется в утилитарные библиотеки.

Код без дубликаций прост для чтения и понимания. Некоторые анализаторы кода отслеживают дублирование кода в программе и выдают ошибку.

Код должен присутствовать в программе в единственном экземпляре.



You ain't gonna need it.

- Принцип требующий ограничивать стремления программистов к созданию универсальных решений. “Универсальные” решения приводит к появлению мертвого кода (Deadcode):
 - Автор кода должен быть уверен, что написанный код необходим здесь и сейчас.
 - Не надо писать код на будущее.
 - В случае ревью кода и рефакторинг, не стоит бояться удалять неиспользуемые методы.
 - В случае если метод в данный момент не нужен, каким бы он ни был полезным он должен быть удален.
- DeadCode сильно засоряет программу, усложняет чтение и отладку. Не всегда понятно является ли текущий кусок кода рабочим или это “просто лишний текст”

Если удаленный код вдруг понадобился, всегда есть git-его история позволит воскресить код

SoC

Separation of Concerns

- Принцип требующий разделения ответственности между различными сущностями. Создание убекклассов или всемогущих усложняет работу с кодом.
- Например в приложениях часто используется несколько уровней (concerns):
 - User interface.
 - Business logic.
 - Storage.
 - ...

Принцип разделения ответственности регламентирует создание различных абстракций (классов) под каждый уровень.

- Такое разделение позволяет безопасно улучшать или заменять различные уровни логики программы без модификации всего остального.



LoD

Law of Demeter

- Закон Деметры или принцип наименьшего знания.
- Принцип запрещает использование неявных зависимостей:
"Объект А не должен иметь возможность получить непосредственный доступ к объекту С, если у объекта А есть доступ к объекту В и у объекта В есть доступ к объекту С".
- Это означает, что все зависимости в коде должны быть явно определены. Взаимодействие между классами/модулями должны осуществляться только через доступные интерфейсы, а не завязываться на знании о внутренней архитектуре друг друга.
- Данный принцип декларирует круг взаимодействия класса/модуля:
 - Со своими методами
 - С методами объектов, от которых объект зависит напрямую.
 - С объектами, переданными в метод в качестве параметра.
 - С глобальными переменными (осторожно, глобальные переменные во многом увеличивают общую сложность кода)

Итак получается, что friends - являются прямым нарушением этого принципа



SOLID

- Принцип SOLID описывает пять ключевых парадигм проектирования и организации объектно-ориентированного кода. Был сформулирован Робертом Мартином (также известным как Uncle Bob) в начале 2000.
- SOLID включают в себя следующие концепции:
 - [S] Single Responsibility,
 - [O] Open-Closed,
 - [L] Liskov Substitution,
 - [i] Interface Segregation
 - [D] Dependency Inversion.
- Цель использования принципов SOLID — упростить разработку, сделать ее более гибкой и устойчивой к ошибкам.



[S]olid: Single Responsibility

Принцип единой ответственности

- «Проектируя классы, мы должны стремиться к тому, чтобы объединять родственные компоненты, то есть такие, изменения в которых происходят по одним и тем же причинам. Нам следует стараться разделять компоненты, изменения в которых вызывают различные причины» Стив Фентон.
- Каждый объект, класс и метод должны отвечать только за что-то одно. Если ваш объект/класс/метод делает слишком много, вы усложните код.
- Принцип схож с SoC, но ограничен применением на уровне одного класса или модуля.

```
class Animal
{
    Animal(const std::string& name);
    std::string getAnimalName();
};

class RedBook
{
    // хорошо. Животное само по себе
    bool saveAnimal(const Animal& animal);
};
```

s[O]lid: Open-Closed

Принцип открытой закрытости.

- Программные объекты должны быть открыты для расширения, но закрыты для модификации. Т.е. нельзя переопределять существующие методы или классы, добавляя в них новые функции по мере необходимости.
- Решения этой проблемы – использование наследования.
- Простое правило: если вы изменяете сущность, чтобы сделать ее расширяемой, вы нарушаете этот принцип

```
class Animal
{
public:
    virtual std::string sound() const = 0;
};

class Lion : public Animal
{
public:
    std::string sound() const override
    {return "roar";};
};

class Mouse : public Animal
{
public:
    std::string sound() const override
    {return "squeak";};
};
// добавление нового животного
// это добавление нового класса.
```

```
std::string animalSound(const std::vector<Animal>& animals)
{
    // добавление нового животного не приведет к изменению кода в этом месте.
    for(const auto& animal : animals)
        animal.sound();
    return "";
}

// ...

return "";
}
```

so[L]id: Liskov Substitution

Принцип подстановки Барбары Лисков

- "Объекты в программе должны быть заменяемыми на экземпляры их подтипов без изменения корректности программы".
- Этот принцип гласит, что объекты старших классов должны быть заменимы объектами подклассов, и приложение при такой замене должно работать так, как ожидается.
- Это означает, что если класс В является подклассом класса А, то мы должны иметь возможность заменить объект А на объект В без каких-либо проблем.

```
std::string animalSound(const std::vector<Animal*>& animals)
{
    // объект в этом цикле можно заменить на объект любого дочернего класса и поведение программы не изменится
    for(const auto& animal : animals)
        animal->sound();
    return "";
}
```


solid: Interface segregation

Принцип разделения интерфейсов

- Принцип говорит о том, что "клиенты не должны зависеть от интерфейсов, которые они не используют". Это означает, что большой интерфейс должен быть разбит на несколько меньших и более конкретных, чтобы клиенты могли использовать только те интерфейсы, которые им нужны.
- Допустим, у нас есть интерфейс Machine с методами print, fax, scan. Но что, если у нас есть простой принтер, который может только печатать? В этом случае, наш принтер вынужден имплементировать методы, которые ему не нужны, что является нарушением принципа разделения интерфейса.

```
struct VoicedAnimal
{
    virtual std::string sound() const = 0;
};
struct Terrestrial
{
    virtual void walk() = 0;
    virtual void run() = 0;
};
struct FlyingAnimal
{
    virtual void fly() = 0;
    virtual void takeOff() = 0;
    virtual void land() = 0;
};
struct WaterAnimal
{
    virtual void swim() = 0;
};

class Duck : public VoicedAnimal,
             FlyingAnimal, WaterAnimal
{
    std::string sound() const override;
    void swim() override;
    void fly() override;
    virtual void takeOff() override;
    virtual void land() override;
};

class Dog : public VoicedAnimal, Terrestrial
{
    std::string sound() const override;
    void walk() override;
    void run() override;
};
```

solli[D]: Dependency inversion

Принцип инверсии зависимостей

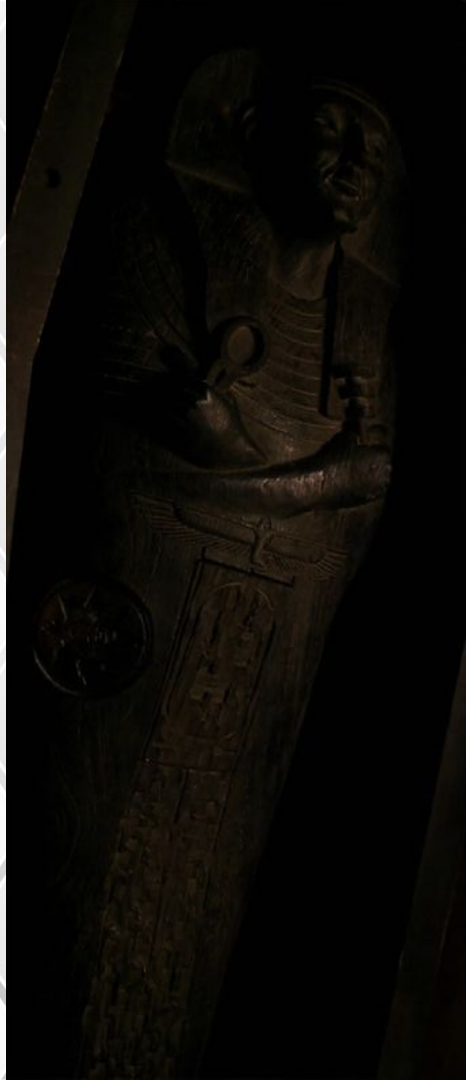
- Принцип утверждает, что "зависимости внутри системы должны строиться на основе абстракций, а не деталей".
- Это означает, что высокоуровневые модули не должны зависеть от конкретных реализаций, а они все должны зависеть от абстракций.
- Заботиться нужно не о том, как что-то устроено, а о том, как оно работает.
- Простой пример – использование дат. Если написать для получения дат слой абстракции, то в случае смены источника получения дат, нужно будет внести изменения только в месте, реализации абстракции с новым источником.

```
// Во всех местах где нужна дата или интервал используем данный класс
// таким образом мы скрываем детали реализации получения даты
class Date
{
    Time getTimeOfDay() const;
    int  getDurationMSec() const;
}
```

Premature Optimization

Преждевременная оптимизация.

- «Преждевременная оптимизация — корень всех зол» Дональд Кнут - автор книги «Искусство программирования»
- Часто разработчики порываются написать крайне оптимальный код или оптимизировать существующий код до того, как необходимость в оптимизации будет доказана. Обоснованность того или иного сложного метода должна быть доказана до его написания.
- Преждевременная оптимизация может привести к задержкам в сроках разработки кода и, следовательно, увеличит затраты времени на вывод функций на рынок. Может привести к резкому падению ROI.
- К оптимизации нужно подходить взвешенно, есть риск пропустить момент, когда оптимизация может быть сделана наиболее эффективно (Рэндал Хайд).



KISS

Keep It Simple, Stupid (short and simple, simple and straightforward, ...)

- Принцип инженерного проектирования. Впервые упомянут в 1938 г., принят в ВМС США в 1960 году. Сформулирован Кларенсом Джонсоном, ведущим инженером Lockheed.
- Качество кода часто измеряется числом строк, объемом проделанной работы или количеством примененных методологий. Последствия такого подхода:
 - Необоснованное усложнение кода,
 - Снижение стабильности кода, увеличенное количество багов,
 - Удорожание разработки,
 - Удорожание поддержки.
- Принцип KISS переносит внимание на результат. Когда программа справляется с поставленной задачей, не важно какие решения в ней реализованы.

Соблюдение принципа KISS, позволяет существенно повысить ROI разработки программного продукта.
Принцип TDD основывается на KISS



A decorative pattern of interlocking hexagons in various shades of gray, arranged in a staggered grid. The pattern is located on the left side of the slide, with some hexagons extending towards the center.

THANK YOU!